Don't Dwindle, Don!: A Dwarven Resource Management Simulator for Teaching Responsible Consumption

Aaron Jared Lee
College of Computer and Information Sciences
IT102L - Object-Oriented Programming
October 2026

***Abstract*—This paper presents "Don't Dwindle, Don!", a resource management simulation game developed using Python and the Streamlit framework. The application addresses the real-world problem of resource mismanagement and scarcity, challenging players to make difficult distribution decisions under pressure. By placing the player in the role of a dwarven leader managing food, water, medicine, gold, and morale, the game demonstrates the cascading consequences of resource allocation. The project incorporates Object-Oriented Programming (OOP) principles, including inheritance, encapsulation, and polymorphism, in order to manage game entities and logic. The resulting application serves as an interactive educational tool aligned with United Nations Sustainable Development Goals (SDGs) 2, 6, and 12, teaching players the importance of sustainable management and long-term planning.**

I. Introduction

In the real world, communities and leaders frequently face the challenge of distributing limited resources (food, water, medicine) among people who need them, often leading to long-term consequences if mismanaged. *Don't Dwindle, Don!* is a web-based simulation game that replicates this dilemma. Players assume the role of Don, a dwarven Resource Manager, who must accept, deny, or reject requests from non-player characters (NPCs) on a daily basis. The game directly supports SDG 2 (Zero Hunger), SDG 6 (Clean Water), and SDG 12 (Responsible Consumption and Production) by simulating scarcity and rewarding sustainable decision-making.

II. Methods

The application was developed using Python and Streamlit, utilizing an Object-Oriented Programming (OOP) architecture.

A. User Journey

The game follows a daily loop. The player views their resources, receives a request from a dwarf, and chooses to accept or reject it. This updates the resources and logs the event in Don's DocuLog. At the end of the day, an evaluation occurs where gold is earned based on morale, and the player can purchase upgrades in the Forge & Market before advancing to the next day.

B. System Architecture

The system is built on five core classes. The `Character` class serves as a parent class for shared attributes (name, role). The `Player` class inherits from Character and manages the hold's resources (food, water, medicine, gold, morale). The `NPC` class also inherits from Character and generates requests. The `Request` class represents a specific resource demand. Finally, the `Game` class acts as the controller, managing the game loop, day progression, and consequence logic.

III. Results and Discussion

The implementation of *Don't Dwindle, Don!* successfully demonstrates the application of OOP principles and complex game logic in a web environment. The game features three main features:

Don's DocuLog

A consequence tracker that logs every decision and its exact morale impact. It displays end-of-day statistics, showing players a breakdown of their accepted, denied, and rejected requests and how much morale was lost or gained.

JSON Save/Load System

To ensure data persistence on the web, the game allows players to download their current game state as a JSON file and upload it later to resume their run.

Butterfly Effect System

The game features a multi-day consequence system. If the hold ends a day with critically low resources or low morale, the next day begins with penalties. Difficulty scales over seven days, culminating in a narrative "Cave Flood Warning" on Day 4, where NPC requests become heavier and require multiple resources, elevating the difficulty towards the end.

IV. Conclusion and Future Work

Don't Dwindle, Don! was successfully developed as a functional Streamlit application that teaches resource management through engaging gameplay. It effectively utilized OOP to create a scalable and modular codebase. For future work, the developers plan to expand the visual assets by replacing emojis with custom character portraits and integrating a more

robust database for permanent, server-side save data. Further iterations will include additional days, random events, and more complex NPC interactions to increase replayability.

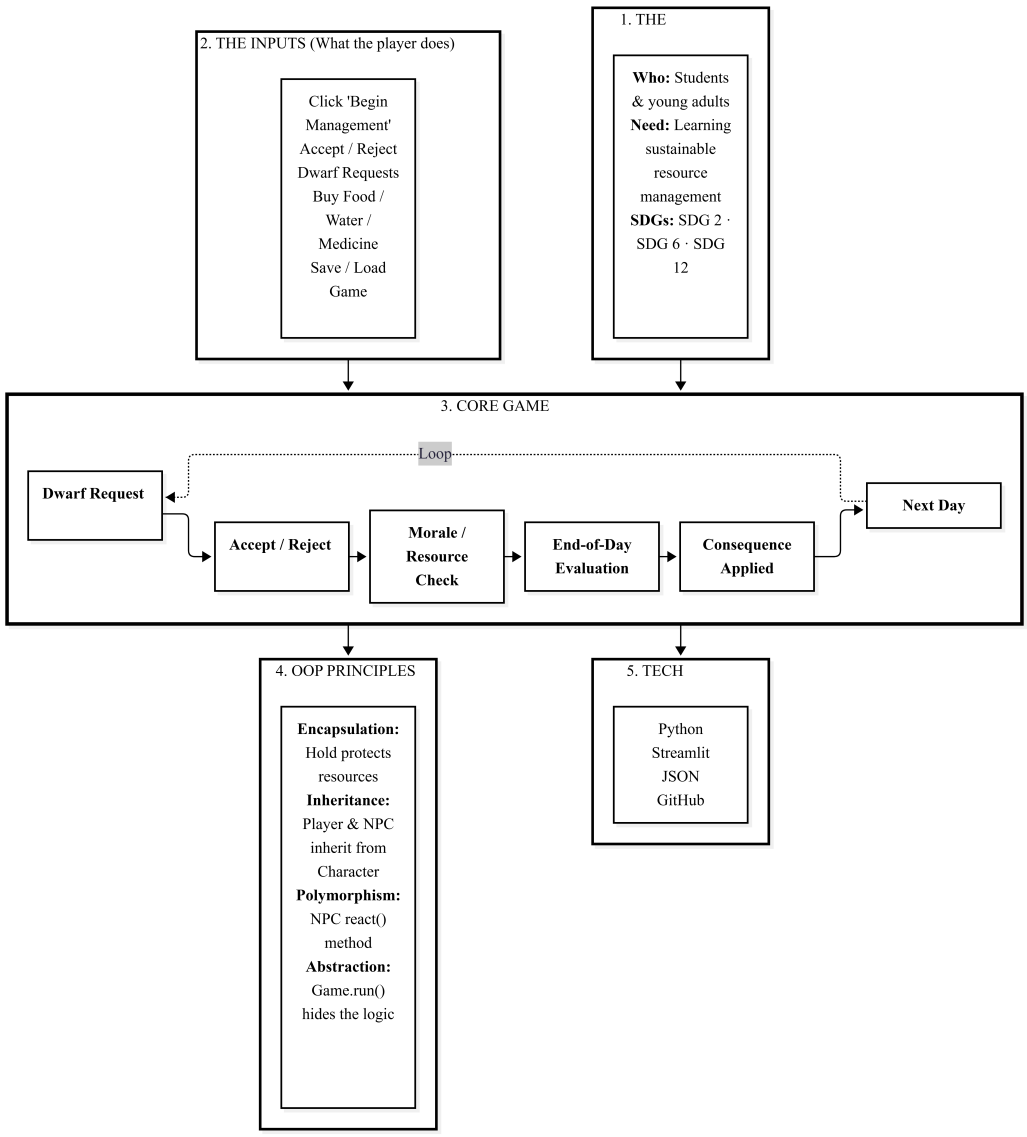


Fig. 1. Conceptual Framework of Don't Dwindle, Don!